

Contents

Volume 04 — Lectures 31–40: Hash Functions, PKI & Network Attacks	1
L31: Cryptographic Hash Functions — Part I	1
L32: Cryptographic Hash Functions — Part II	2
L33: Digital Signature and Certificate	3
L34: Applications — Part I	3
L35: Applications — Part II	4
L36: Steganography	5
L37: Biometrics	6
L38: Network Based Attacks — Part I	7
L39: Network Based Attacks — Part II	8
L40: DNS and Email Security	8

Volume 04 — Lectures 31–40: Hash Functions, PKI & Network Attacks

Ethical Hacking · NPTEL 106105217 · Prof. Indranil Sengupta

L31: Cryptographic Hash Functions — Part I

Concepts

- **Hash function:** maps arbitrary-length input to fixed-length digest; one-way, deterministic.
- **Properties:** preimage resistance, second-preimage resistance, collision resistance.
- **MD5:** 128-bit output; **broken** for collision attacks (do not use for security).
- **SHA-1:** 160-bit output; deprecated (SHAttered collision demonstrated 2017).
- **SHA-2 family:** SHA-256, SHA-384, SHA-512; current standard (NIST FIPS 180-4).

Tools

- `echo -n "data" | sha256sum`
- `openssl dgst -sha256 file`
- `hashcat -m 1400` — crack SHA-256 hashes
- `md5sum`, `sha1sum` — compute digests

Attack Steps

1. Identify hash algorithm from digest length (MD5=32 hex, SHA-1=40, SHA-256=64).
2. Rainbow table / dictionary attack on unsalted hashes.
3. MD5/SHA-1 collision attacks for certificate/document forgery.
4. Length extension attack on MD5/SHA-1 in MAC constructions.

Defenses

- Use SHA-256 or SHA-3 for integrity verification
- **Salt** passwords before hashing (unique per user)
- Use HMAC or bcrypt/scrypt/Argon2 for password storage

- Deprecate MD5 and SHA-1 in all security contexts

Exam Bullets

- Hash function properties: **one-way, deterministic, collision-resistant**.
 - MD5 output = **128 bits** (32 hex chars); cryptographically broken.
 - SHA-256 output = **256 bits** (64 hex chars); current standard.
 - Salting prevents **rainbow table** attacks on password hashes.
 - Preimage resistance: given hash, cannot find original input.
-

L32: Cryptographic Hash Functions — Part II

Concepts

- **SHA-3 (Keccak)**: sponge construction; different internal design from SHA-2 (safety net).
- **HMAC**: Hash-based MAC = $H(K \parallel \text{opad}, H(K \parallel \text{ipad}, \text{message}))$; provides integrity + authentication.
- **Password hashing**: bcrypt, scrypt, Argon2 — deliberately slow + salted (resist brute-force).
- **Merkle-Damgård construction**: iterative hashing used by MD5, SHA-1, SHA-2.
- **Birthday attack**: find collision in $\sim 2^{(n/2)}$ operations for n-bit hash.

Tools

- `openssl dgst -sha3-256 file`
- `htpasswd -nbB user password` — bcrypt hash
- Python: `bcrypt.hashpw()`, `argon2.PasswordHasher()`
- `hashcat -m 3200` — bcrypt cracking

Attack Steps

1. Crack bcrypt hashes with hashcat (slow — high cost factor helps defense).
2. Birthday attack on truncated hashes (e.g., 64-bit truncated SHA-256).
3. Length extension on Merkle-Damgård MACs without HMAC wrapper.
4. GPU-accelerated brute-force on unsalted SHA-256 password hashes.

Defenses

- Argon2id for password storage (winner of Password Hashing Competition)
- HMAC-SHA256 for message authentication codes
- Full hash output (no truncation below 128 bits)
- Rate limiting on authentication endpoints

Exam Bullets

- Birthday attack complexity: $2^{(n/2)}$ for n-bit hash.
- HMAC = hash + secret key → provides **authentication + integrity**.
- bcrypt/scrypt/Argon2 are **adaptive** — configurable work factor.
- SHA-3 uses **sponge construction** (not Merkle-Damgård).
- Never store passwords as plain SHA-256 — always use **salt + slow hash**.

L33: Digital Signature and Certificate

Concepts

- **Digital signature:** hash of message encrypted with sender's private key; proves origin + integrity.
- **Verification:** decrypt signature with public key → compare hash of received message.
- **X.509 certificate:** binds public key to identity; signed by Certificate Authority (CA).
- **PKI hierarchy:** Root CA → Intermediate CA → End-entity certificate.
- **Certificate fields:** subject, issuer, validity period, public key, serial number, extensions.

Tools

- `openssl req -new -x509 -key private.pem -out cert.pem -days 365`
- `openssl verify -CAfile ca.pem cert.pem`
- `openssl x509 -in cert.pem -text -noout`
- Browser certificate viewer (inspect TLS cert chain)

Attack Steps

1. Exploit weak CA practices (issue cert for domain you don't own).
2. Self-signed cert MITM if victim accepts/trusts attacker cert.
3. Certificate transparency logs to find misissued certs.
4. Expired/revoked cert acceptance if client doesn't check OCSP/CRL.

Defenses

- Use trusted public CAs (Let's Encrypt, DigiCert)
- Certificate pinning (HPKP deprecated; use Expect-CT, CAA records)
- OCSP stapling for revocation checking
- Short certificate validity (90 days — Let's Encrypt standard)

Exam Bullets

- Digital signature uses sender's **private key**; verified with **public key**.
- X.509 certificate binds **identity to public key**, signed by CA.
- PKI chain: Root CA → Intermediate → End-entity.
- Self-signed certs are **not trusted** by default in browsers.
- OCSP checks if certificate has been **revoked** before expiry.

L34: Applications — Part I

Concepts

- **SSL/TLS:** secure transport layer; versions TLS 1.2 and 1.3 current; SSLv3/TLS 1.0/1.1 deprecated.
- **TLS handshake:** ClientHello → ServerHello → Certificate → Key Exchange → Finished.

- **Cipher suite:** key exchange + authentication + bulk encryption + MAC (e.g., TLS_AES_256_GCM_SHA384).
- **Perfect Forward Secrecy (PFS):** ephemeral DH keys — compromise of long-term key doesn't decrypt past sessions.
- **IPsec:** network-layer security; modes: Transport (host-to-host) and Tunnel (gateway-to-gateway).

Tools

- `openssl s_client -connect host:443 -tls1_2`
- `nmap --script ssl-enum-ciphers target`
- `testssl.sh` — comprehensive TLS testing
- `sslyze` — TLS configuration analyzer

Attack Steps

1. Enumerate supported cipher suites: `nmap --script ssl-enum-ciphers`.
2. Exploit weak ciphers (RC4, 3DES, export-grade).
3. POODLE attack on SSLv3 (CBC padding oracle).
4. Downgrade attack forcing TLS 1.0 with weak ciphers.

Defenses

- Enforce TLS 1.2+ only; prefer TLS 1.3
- Disable weak ciphers (RC4, 3DES, NULL, EXPORT)
- Enable HSTS header: `Strict-Transport-Security: max-age=31536000`
- Use ECDHE cipher suites for forward secrecy

Exam Bullets

- TLS provides **confidentiality + integrity** at transport layer.
- TLS 1.3 reduces handshake to **1-RTT** (faster than TLS 1.2's 2-RTT).
- PFS ensures past sessions safe even if **long-term key compromised**.
- IPsec **Tunnel mode** encrypts entire IP packet (VPN use case).
- Cipher suite specifies: key exchange, authentication, encryption, MAC algorithms.

L35: Applications — Part II

Concepts

- **PGP/GPG:** email encryption and signing; web of trust model (vs PKI hierarchy).
- **S/MIME:** email encryption using X.509 certificates (enterprise standard).
- **VPN protocols:** IPsec (IKEv2), OpenVPN (SSL/TLS), WireGuard (modern, lightweight).
- **SSH:** secure remote shell; key-based auth preferred over passwords.
- **Kerberos:** network authentication using tickets (Active Directory).

Tools

- `gpg --encrypt --sign -r recipient@email.com file`

- `gpg --verify signature.asc file`
- `ssh -i key.pem user@host`
- `openvpn --config client.ovpn`

Attack Steps

1. Steal GPG private key or SSH private key from compromised host.
2. Kerberoasting: request service tickets and crack offline (Active Directory).
3. VPN credential brute-force on exposed endpoints.
4. SSH weak key exchange algorithms (diffie-hellman-group1-sha1).

Defenses

- SSH key-based auth with Ed25519 keys; disable password auth
- GPG key passphrase protection; hardware token (YubiKey)
- Kerberos AES encryption; monitor for TGS-REQ anomalies
- VPN MFA + certificate-based authentication

Exam Bullets

- PGP uses **web of trust**; S/MIME uses **PKI/X.509**.
 - SSH default port = **22**; key-based auth more secure than passwords.
 - Kerberos uses **tickets** (TGT + service tickets) for authentication.
 - OpenVPN tunnels traffic over **SSL/TLS** (port 443/UDP 1194).
 - WireGuard is a modern VPN using **Curve25519** key exchange.
-

L36: Steganography

Concepts

- **Steganography**: hiding secret data within innocuous carrier (image, audio, video, text).
- **LSB (Least Significant Bit)**: replace least significant bits of pixel values with secret bits.
- **Steganalysis**: detecting hidden data via statistical analysis (chi-square, RS analysis).
- **Tools hide data in**: image formats (BMP, PNG), audio (WAV), documents (PDF).
- **Difference from encryption**: steganography conceals **existence**; encryption conceals **content**.

Tools

- `steghide embed -cf image.jpg -ef secret.txt`
- `steghide extract -sf image.jpg`
- `stegsolve` — visual steganalysis
- `binwalk` — extract embedded files from images/firmware
- `zsteg` — LSB steganography detection (PNG/BMP)

Attack Steps

1. Statistical analysis: chi-square test on LSB distribution.
2. Visual analysis with Stegsolve (bit planes, color channels).

3. `binwalk -e image.png` to extract embedded files.
4. Compare original vs suspect file sizes and entropy.

Defenses

- Re-encode images (destroys LSB steganography)
- DLP scanning for anomalous file entropy
- Monitor outbound file transfers for unusual patterns
- Hash comparison of known-clean vs suspect media

Exam Bullets

- LSB steganography modifies **least significant bits** of pixel values.
 - Steganography hides message **existence**; cryptography hides **content**.
 - `steghide` supports JPEG and BMP with optional passphrase encryption.
 - Chi-square test detects **non-random LSB distribution** (steganalysis).
 - `binwalk` identifies and extracts embedded files by signature scanning.
-

L37: Biometrics

Concepts

- **Biometric modalities:** fingerprint, iris, face, voice, gait, palm print.
- **Performance metrics:** FAR (False Accept Rate), FRR (False Reject Rate), EER (Equal Error Rate).
- **Liveness detection:** distinguish live person from photo/mold/replay attack.
- **Template storage:** store feature vectors, not raw images; irreversible transforms preferred.
- **Multimodal biometrics:** combine two+ modalities for higher accuracy.

Tools

- OpenCV + dlib (face recognition)
- `fingerprint` SDK (vendor-specific)
- Presentation Attack Detection (PAD) frameworks

Attack Steps

1. **Spoofing:** gelatin fingerprint, printed photo, 3D face mask.
2. **Database theft:** steal biometric templates (cannot change biometrics like passwords).
3. **Replay attack:** capture and replay biometric signal.
4. **Morphing attack:** combine two faces in passport photo.

Defenses

- Liveness detection (blink, pulse, challenge-response)
- Cancelable biometrics (revocable templates)
- Multimodal fusion (fingerprint + iris)
- Store templates with homomorphic encryption or secure enclave

Exam Bullets

- FAR = unauthorized user **accepted**; FRR = authorized user **rejected**.
 - EER = point where FAR equals FRR (lower is better).
 - Biometric templates **cannot be changed** if compromised (unlike passwords).
 - Liveness detection prevents **presentation attacks** (spoofing).
 - Multimodal biometrics improves accuracy by combining modalities.
-

L38: Network Based Attacks — Part I

Concepts

- **Sniffing**: capture packets on network segment (promiscuous mode).
- **Spoofing types**: IP spoofing, MAC spoofing, email spoofing, DNS spoofing.
- **Session hijacking**: take over established TCP session using stolen seq/ack numbers.
- **TCP RST attack**: send forged RST to tear down connection.
- **DNS cache poisoning**: inject false DNS records into resolver cache.

Tools

- Wireshark / `tcpdump -i eth0`
- `macchanger -m random eth0`
- `hping3 --rst` — send TCP RST packets
- `dnsspoof (dsniff)` — DNS spoofing

Attack Steps

1. Enable promiscuous mode; capture traffic on shared segment (or after ARP spoof).
2. Filter for credentials: Wireshark `http.request.method == "POST"`.
3. DNS cache poisoning: flood resolver with spoofed responses (Kaminsky attack).
4. TCP session hijack: predict seq numbers (older stacks) or after MITM position.

Defenses

- Switches (not hubs) + port security
- Encrypt traffic (TLS everywhere)
- DNSSEC prevents cache poisoning
- Randomized TCP ISN (initial sequence numbers)

Exam Bullets

- Promiscuous mode captures **all frames** on segment (not just addressed to host).
 - DNS cache poisoning injects **false records** into resolver cache.
 - TCP session hijacking requires knowing/c predicting **sequence numbers**.
 - IP spoofing: forge source IP (used in DDoS reflection attacks).
 - Switches limit sniffing to own port traffic (unless ARP spoofed or MAC flooded).
-

L39: Network Based Attacks — Part II

Concepts

- **Firewall types:** packet-filtering (stateless), stateful inspection, application-layer (proxy).
- **IDS (Intrusion Detection):** passive monitoring, alerts on suspicious patterns (Snort, Suricata).
- **IPS (Intrusion Prevention):** active blocking of detected threats.
- **Detection methods:** signature-based (known patterns) vs anomaly-based (baseline deviation).
- **Evasion techniques:** fragmentation, encoding, timing attacks, encrypted payloads.

Tools

- `iptables -L -n -v` — Linux firewall rules
- Snort / Suricata — IDS/IPS
- `nmap -f -D RND:5` — fragmented decoy scan
- `proxychains` — route through proxy to evade detection

Attack Steps

1. Identify firewall type via TTL analysis and port probing.
2. Fragment packets to evade signature-based IDS: `nmap -f`.
3. Use slow scan timing: `nmap --scan-delay 5s`.
4. Tunnel attack traffic through allowed ports (DNS tunneling, HTTPS).

Defenses

- Stateful firewall with default-deny policy
- IDS/IPS with updated signature sets
- Deep packet inspection and reassembly
- Network segmentation and zero-trust architecture

Exam Bullets

- Stateful firewall tracks **connection state** (unlike stateless packet filter).
 - IDS = **detect and alert**; IPS = **detect and block**.
 - Signature-based IDS matches **known attack patterns** (Snort rules).
 - Anomaly-based IDS detects **deviations from baseline** behavior.
 - Fragmentation evades IDS that doesn't **reassemble packets** before inspection.
-

L40: DNS and Email Security

Concepts

- **DNS record types:** A, AAAA, MX, NS, TXT, CNAME, PTR, SOA.
- **DNSSEC:** cryptographically signs DNS records (RRSIG, DNSKEY, DS); prevents spoofing.
- **Email security protocols:** SPF (authorized senders), DKIM (signed messages), DMARC (policy enforcement).

- **SPF:** TXT record listing permitted sending IPs for domain.
- **DKIM:** digital signature in email header; verified against public key in DNS TXT.
- **DMARC:** policy telling receivers what to do with SPF/DKIM failures (none/quarantine/reject).

Tools

- `dig domain.com TXT` — check SPF/DKIM records
- `dig domain.com DNSKEY` — DNSSEC keys
- `swaks --to victim@domain.com --from attacker@spoofed.com` — email testing
- <https://mxtoolbox.com> — email/DNS diagnostics

Attack Steps

1. **Email spoofing:** send mail from forged From address (if SPF/DKIM/DMARC absent).
2. **Phishing:** spoof legitimate sender domain with lookalike domain.
3. **DNS hijacking:** compromise registrar account; change NS records.
4. **Subdomain takeover:** claim abandoned CNAME target (e.g., deleted S3 bucket).

Defenses

- Implement SPF + DKIM + DMARC (p=reject)
- Enable DNSSEC on all zones
- Monitor DMARC aggregate reports
- Lock domain registrar account with MFA

Exam Bullets

- SPF = **authorized sending IP addresses** for a domain.
- DKIM = **cryptographic signature** on email headers/body.
- DMARC = **policy** for handling SPF/DKIM failures.
- DNSSEC prevents **DNS response spoofing** via digital signatures.
- MX record specifies **mail server** for domain.

Previous: [Volume 03](#) · Next: [Volume 05 — Passwords, Malware & Hardware Security](#)